2. Linguagem de Programação C++

2.1 Fundamentos de Programação de Computadores

☞ O que é um Programa de Computador?

A utilização do computador para a resolução de problemas de forma automática traz grandes vantagens:

- ✓ **Rapidez de execução;**
- ✓ **Precisão nos processamentos e resultados;**
- ✓ **Flexibilidade;**
- ✓ **Confiança nos resultados obtidos; etc.**

Entretanto, o computador por si só não seria capaz de resolver o mais simples dos problemas que nos possa ocorrer.

Para que ele o faça, é preciso descrever com detalhes em uma linguagem que ele entenda (que são as linguagens de programação de computadores) todos os passos que devem ser executados para a resolução do problema. Esta descrição recebe o nome de ***PROGRAMA DE COMPUTADOR***.

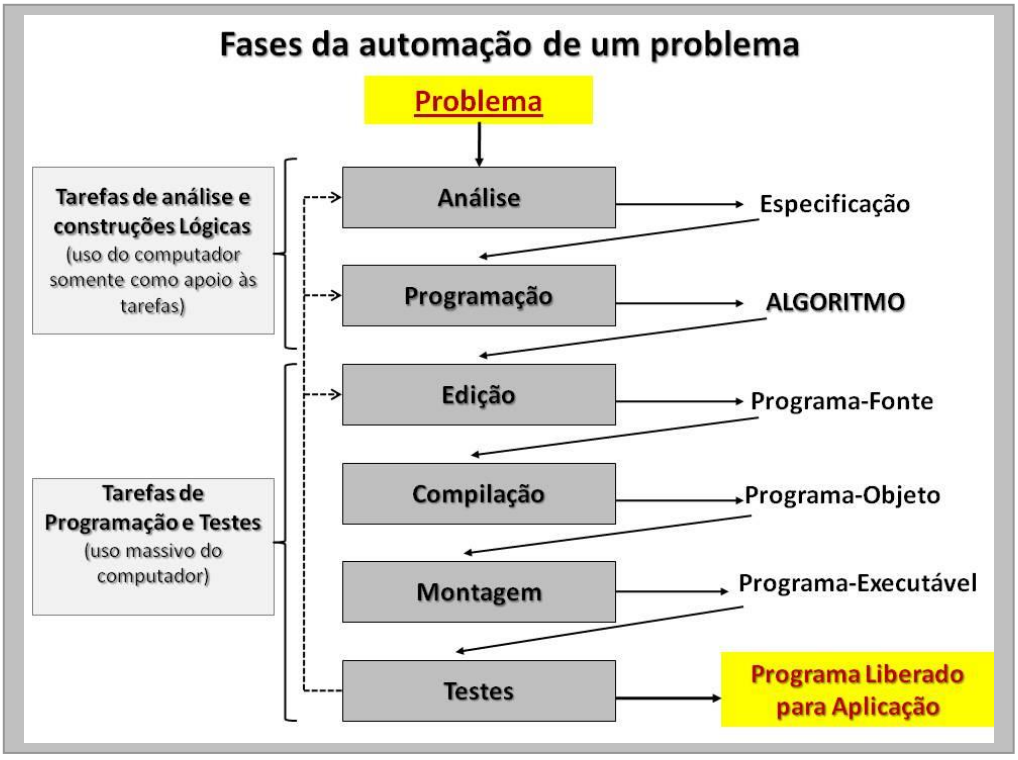
Basicamente a função de um programa é dirigir o funcionamento do computador.

Portanto, para que os computadores possam atuar de forma eficiente em todas as áreas, eles precisam ser **PROGRAMADOS** também de forma eficiente.

✓ **PROGRAMA DE COMPUTADOR:** consiste em um conjunto de instruções que específica como os dados, inicialmente fornecidos, serão transformados nas informações desejadas.

A função básica de um Programa de Computador é dirigir e controlar o funcionamento do computador, fazendo com que ele resolva problemas de forma automática.

☞ Fases da automação de um problema ou Etapas de Programação de Computador



A figura acima mostra as fases da resolução de um problema computacional, utilizando a programação de computador, que são:

a) **Análise e compreensão do problema, <u>especificando</u> uma (ou mais) estratégia para sua resolução**;

b) A **preparação de um algoritmo** (**<u>programação</u>**), baseado na melhor estratégia para resolução do problema;

c) A **transformação desse algoritmo em um programa de computador** (**<u>edição</u>**), utilizando um simples editor de texto, como por exemplo o Bloco de Notas do MS-Windows®. Todavia, todos os ambientes de desenvolvimento de programas (IDE - *Integrated Development Environment*), possuem um editor de textos.

   Nesta disciplina será utilizada a IDE Dev-C++, que além do compilador da linguagem C++, disponibiliza um Editor de Textos;

d) A **<u>compilação</u>, <u>montagem</u> e geração de um programa executável;**

e) **A execução desse programa, para avaliar sua precisão e potencial de resolução do problema proposto** (**<u>testes</u>**) e também de outros problemas.

As etapas **a)** e **b) não necessariamente precisam de um computador** para serem desenvolvidas, pois para escrever algoritmos basta lápis e papel, porém as demais etapas só podem ser desenvolvidas usando-se o computador e a IDE para programação da solução para o problema.

Dado um determinado problema a ser revolvido utilizando computadores, é altamente recomendável que o processo de desenvolvimento do programa que irá solucionar o problema siga as fases acima mencionadas.

☞ Fundamentos de Linguagem de Programação

Linguagem - é o sistema de sinais, que empregamos para exprimir nossas idéias e pensamentos. É a expressão do pensamento por meio da palavra.

Linguagem de Computador ou de Programação - é uma notação (sistema de sinais) para escrever programas, através dos quais podemos nos comunicar com o computador e seus dispositivos periféricos e dar assim as ordens adequadas para realização de uma determinada tarefa. É a forma codificada de escrever instruções para o computador.

Uma <u>linguagem de computador</u> é composta por um conjunto de símbolos, palavras e comandos, governados por <u>regras fixas</u> (sintaxe).

A linguagem de programação de computador é o instrumento de comunicação do homem com a máquina, e está cada vez mais próxima da expressão verbal humana.

☞ Classificação de Linguagem de Computador

✓ <u>Linguagem de máquina - (década de 40)</u>

É a linguagem que o computador entende de forma direta. Utiliza o alfabeto binário, expresso em **0-desligado** e **1-ligado** (**base binária**). Essa programação era feita usando-se chaves no painel do computador.

✓ <u>Linguagem de baixo nível</u> - (<u>inicio da década de 50)</u>

A linguagem de baixo nível, ou linguagem de montagem, é estruturalmente a linguagem mais próxima da máquina.

É uma linguagem que associa mnemônicos às instruções em linguagem de máquina.

Exemplo: Linguagem ASSEMBLY.

```
INICIO:  ADD     B,1
         MOV     A,B
         CMP     A,E
         JE      FIM
         JMP     INICIO
FIM:     END
```

✓ **Linguagem de alto nível – (final da década de 50)**

É a linguagem que possui comandos poderosos que tornam a atividade de programar fácil e rápida. **É a quando a linguagem está mais próxima do programador.** Nesse tipo linguagem ocorre um distanciamento entre a estrutura do programa e a estrutura do hardware que o executa.

Exemplos de linguagens de alto nível: **FORTRAN, BASIC, Pascal, COBOL, PLI, C, CLIPPER, etc.**

As linguagens de alto nível vêm evoluindo ao longo do tempo, originando outras como o C++, C#, Phyton, Object Pascal, Java, etc., sendo que muitas delas incorporaram os conceitos da POO (Programação Orientada por Objetos).

✓ **Linguagem de Inteligência artificial- (década de 80)**

É a linguagem que está mais próxima da linguagem humana que as linguagens de alto nível típicas. Linguagem de IA é usada, entre outros, no desenvolvimento de sistemas especialistas, acesso à banco de dados e desenvolvimento de aplicativos genéricos e alguns específicos.

☞ Conceitos sobre Montadores, Interpretadores e Compiladores

A programação de computadores, feita diretamente em linguagem de máquina exigia profundos conhecimentos de lógica e hardware, num nível de que poucos profissionais dispunham.

Com a finalidade de se ter alternativa à até então obrigatória programação em linguagem binária, desenvolveram-se novas formas de transformar automaticamente instruções simbólicas e verbais em instruções binárias que são as reconhecidas pelo computador.

✓ Montadores

A linguagem de montagem tem como característica transformar uma instrução simbólica (mnemônicos) em instrução binária (linguagem de máquina). Os montadores foram desenvolvidos para eliminar o trabalho tedioso de programar os computadores diretamente em linguagem de máquina.

✓ Interpretadores

Os interpretadores efetuam a tradução da linha de comando de um programa fonte, em linguagem de máquina, no momento da execução do programa. Esse procedimento é efetuado linha a linha do programa fonte.

✓ Compiladores

São programas que traduzem um programa, escrito em alguma linguagem de programação (código fonte), para linguagem de máquina (código objeto), sendo que após um processo denominado link-edição, será gerado efetivamente o programa executável (linguagem de máquina).

A execução de um programa compilado é mais rápida que a execução de um programa interpretado.

2.2 Histórico da linguagem de programação C

A Linguagem C, foi criada em 1970 por Dennis Ritchie, sendo uma evolução da Linguagem B que, por sua vez, foi uma adaptação feita por Ken Thompson, a partir da Linguagem BCPL (*Basic Combined Programming Language*) criada em 1966 por Martin Richards, um cientista da computação britânico, da Universidade de Cambridge.

Dennis Ritchie e Ken Thompson, ambos engenheiros elétricos e da computação, da Universidade da Califórnia, Berkeley, USA, trabalharam no Bell Labs *(Bell Telephone Laboratories)*, onde criaram o sistema operacional Unix, na década de 70. Porisso, a linguagem C é estreitamente associada a esse sistema operacional, já que as versões atuais do próprio sistema foram desenvolvidas utilizando-se esta linguagem. Originalmente, o sistema operacional UNIX foi desenvolvido usando 70% a linguagem Assembler e 30% a linguagem C, atualmente, devido a evolução da linguagem C desde a década de 70, as versões atuais do Unix estão desenvolvidas usando cerca de 70% da linguagem C e 30% de Assembler, ou seja, houve uma inversão nas atualizações desse sistema.

Devido ao crescente uso e interesse da comunidade de computação pela linguagem C, em 1983, o ANSI (*American National Standards Institute*), estabeleceu um comitê para prover uma definição moderna e abrangente da linguagem C. Como resultado dos trabalhos dessa comissão publicou-se uma padronização dessa linguagem, que passou a ser denominada ANSI-C em 1988.

A maior contribuição da ANSI com esse trabalho foi a incorporação de uma biblioteca padrão, presentes em todas as variações/versões da linguagem, que fornece funções de acesso ao sistema operacional, entrada e saída formatada, alocação de memória, manipulação de strings (cadeias de caracteres), etc.

A Linguagem C é uma **linguagem de programação compilada**, cujo conceito está explicitado no item anterior deste material de aula. É uma das linguagens de programação mais populares e existem poucas arquiteturas computacionais para as quais não existem compiladores para C. Em termos de paradigma de linguagem, o C é uma linguagem imperativa e "procedural", para implementação de sistemas.

Seus principais pontos de design foram:

- ✓ Ser uma linguagem compilada;
- ✓ Fornecer acesso de baixo nível à memória;
- ✓ Possuir baixos requerimentos do *hardware;*
- ✓ Ser, também **uma linguagem de alto nível**, para maior reaproveitamento do código.

☞ Linguagem C - Conceitos básicos

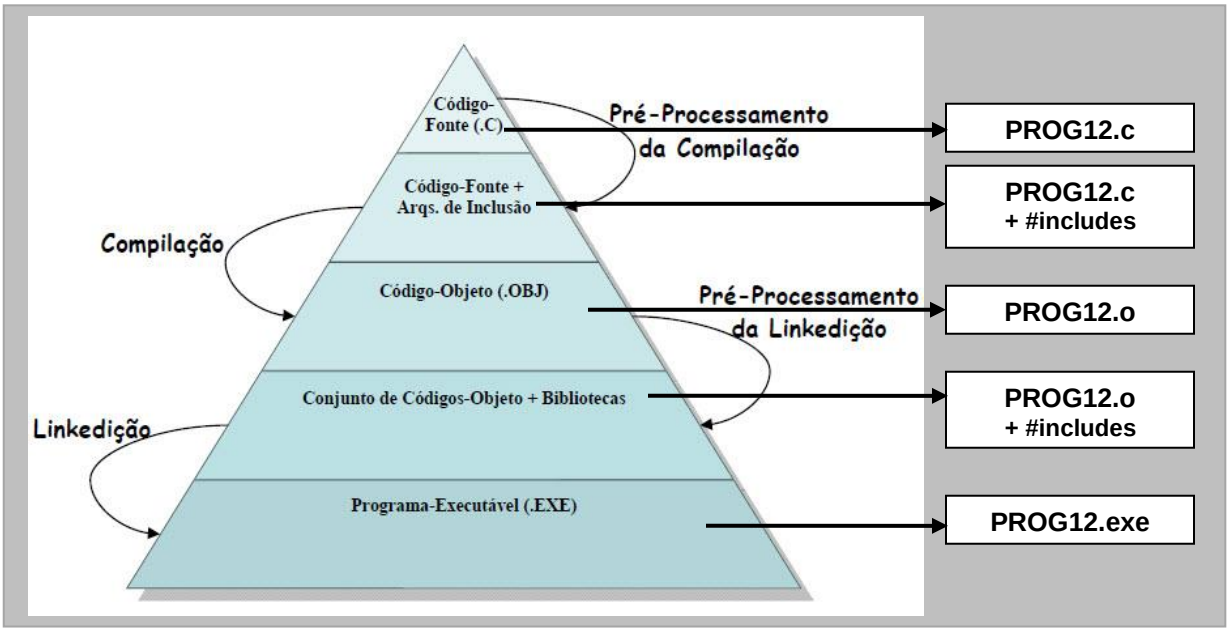
A filosofia básica da linguagem C é que os programadores devem estar cientes do que estão fazendo, ou seja, devem estar cientes daquilo que estão instruindo o computador a fazer e, diante disso, tornem claras e explicitas suas instruções. Agindo dessa forma, os programadores poderão aproveitar-se da elegância e flexibilidade que as linguagens de alto nível proporcionam, (ex: suporte ao conceito de tipo de dados), e do poderio das linguagens de baixo nível (ex: manipulação de bits, bytes e endereços). Portanto, podem desenvolver programas eficazes e eficientes, muito bem documentados.

Pode-se dizer que essa linguagem de programação é de finalidade geral, sendo utilizada no desenvolvimento de diversos tipos de aplicação, como processadores de texto, sistemas operacionais, sistemas de comunicação, programas para solução de problemas de engenharia, física, química, e outras ciências.

Um código-fonte de um programa C (algoritmo nessa linguagem) pode ser escrito usando-se qualquer editor de texto que seja capaz de gerar arquivos em código ASCII (sem formatação). Como o ambiente de programação utilizado pela linguagem C é voltado para o sistema operacional DOS, então, os arquivos contendo os códigos-fontes em C obedecer as regras de

nomeação de arquivos em DOS, ou seja, jamais usar acentuações, e procurar dar nome de no máximo 8 caracteres, e a extensão “c” (exemplo: PROG1234.C), embora na IDE usada pela linguagem C, seja permitido nome de arquivos com mais de 8 caracteres.

Após a digitação do programa-fonte em linguagem C, o mesmo pode ser submetido aos processos de **compilação** e ***linkedição*** (montagem) para geração do programa executável (com extensão “exe”). A figura a seguir representa as etapas do processo de compilação.



Durante o **processo de compilação**, cada programa-fonte com a extensão **".c"** é compilado individualmente, **gerando um arquivo de programa-objeto** com a extensão **“.o”**. Um programa-objeto contêm instruções em linguagem de máquina (códigos binários) entendidas somente pelos microprocessadores.

No processo final da compilação, que é o de *linkedição* (ou ligação), o programa-objeto mais as bibliotecas declaradas no programa-fonte (#include<????>) são processados em conjunto (ligadas), visando a **produção do programa- executável** **".exe"** correspondente.

A seguir, exemplos de bibliotecas da linguagem C, que podem ser incluídas em um programa-fonte.

ARQUIVO	FINALIDADE
assert.h	Contém a macro “assert” que é usada para incluir diagnóstico em programas
ctype.h	Declara funções para testar caracteres
errno.h	Define o número de erro do sistema
float.h	Define os valores em ponto flutuante específicos do compilador
limits.h	Define tipos de valores-limites específicos do compilador
locale.h	Utilizada para adaptar as diferentes convenções culturais
math.h	Define as funções matemáticas e macros usadas para operações matemáticas em linguagem C
setjmp.h	Provê uma forma de evitar a seqüência normal de chamada e retorno de função profundamente aninhada
signal.h	Provê facilidades para manipular condições excepcionais que surgem durante a execução, como um sinal de interrupção de uma fonte externa ou um uso na execução
stdarg.h	Define macros para acessar argumentos quando uma função usa um número variável de argumentos
stddef.h	Define funções, constantes, macros e estruturas usadas para operações padrões de entrada e saída
stdio.h	Contém funções, tipos e macros de entrada e saída
stdlib.h	Contém funções para conversão de números, alocação de memória e tarefas similares
string.h	Define funções para manipulação de *“strings”*
time.h	Define funções e estruturas usadas na manipulação de data e hora.

Durante o processo de compilação, todos os arquivos (programas) gerados possuem o mesmo nome do programa-fonte. Entretanto, quando desejado, o usuário poderá definir diferentes nomes para cada tipo de arquivo.

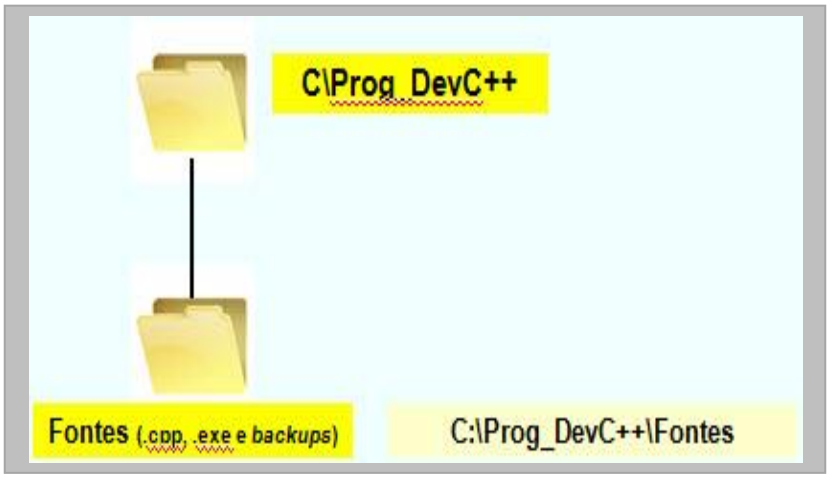
☞ **Sugestão de Estrutura de Diretórios para programas em C**

Quando estamos programando, em qualquer linguagem, é importante criarmos uma estrutura de diretório (pastas), para armazenar os programas que são gerados no ambiente de programação, conforme mostrados anteriormente.

Em um **ambiente de desenvolvimento de programas em linguagem C++** (evolução da linguagem C), que será usado nesta disciplina **(Dev-C++)** para desenvolvimento e testes dos algoritmos previamente desenvolvidos, são gerados os seguintes tipos de arquivos:

- ✓ Programa-fonte com extensão: **.cpp**
- ✓ Cópia de segurança do programa-fonte **(*backup*):** possui a mesma extensão do programa-fonte, porém o **nome do arquivo recebe um número aleatório** indicando que o mesmo é uma cópia do programa-fonte, antes da sua última alteração.
- ✓ Programa-objeto com extensão: **.o**
- ✓ Programa-executável com extensão: **.exe**

Para armazenar esses arquivos, e para facilitar sua recuperação para estudos e exercícios, sugere-se a criação da seguinte estrutura:



2.3 Sobre a Linguagem C++

Conforme já descrito anteriormente, a linguagem C se tornou uma das linguagens de programação mais utilizada, por ser flexível e poderosa. Essa linguagem é a responsável pela criação de milhares de aplicativos (*software*), muitos deles tornaram-se famosos, como o sistema Unix. Além disso, a linguagem C tem sido uma base para o desenvolvimento de aplicativos do tipo "*games*".

Todavia, a linguagem C possui limites quando o tamanho de um projeto (conjunto de programas) ultrapassa a casa de 25.000 a 100.00 linhas de código-fonte. Para solucionar esse, e outros problemas, o cientista da computação dinamarquês Bjarne Stroustrup faz, em 1980, várias intervenções na linguagem C, acrescentando várias intenções (ações propositais), e dá origem à nova linguagem C, denominando-a inicialmente de "C com classes", e somente **por volta dos anos 1983 é que o nome da linguagem foi alterada para linguagem C++**.

2.4 Estrutura de um programa em Linguagem C++

Ao estudar uma linguagem de programação qualquer, é muito importante entender como se estrutura (se escreve) um programa nessa linguagem.

Um programa escrito em linguagem de programação C++ deve seguir a seguinte estrutura básica:

```
Diretivas de compilador
Declarações de variáveis/constantes globais;
Prototipação de funções;
    tipo_dado func1(lista_parametros)
      {
          Bloco de comandos;
      }
    tipo_dado func2(lista_parametros)
      {
          Bloco de comandos;
      }
                  •
                  •
                  •
    tipo_dado funcN(lista_parametros)
      {
          Bloco de comandos;
      }

tipo_dado main(lista_parametros)        // Função Principal e Obrigatória
    {
      Declarações de variáveis/constantes locais;
      Comandos;
      Estruturas de controle (seleção e/ou repetição);
      Comentários;
      Chamada a outras funções;
    }
```

Corpo do Programa Principal

Exemplo da estrutura básica de um programa-fonte em linguagem C++:

☞ **Exemplo de um programa-fonte em linguagem C++**

```
#include<stdio.h>
#include<stdlib.h>

main()

{
   // Definição de variáveis

   int numero = 120;
   float valor1 = 389;
   float valor2 = 756.58769;

   printf("\n----- MEU PRIMEIRO PROGRAMA -----");

   printf("\n\n Numero = %d",numero);

   printf("\n\n valor1 = %f  valor1 + valor2 = %.2f",valor1, valor1 + valor2);

   printf("\n\n valor2 = %.3f",valor2);

   printf("\n\n \x03");

   printf("\n");

   return 0;
}
```

2.5 Algumas características da Linguagem C++

- ✓ O **ponto-e-vírgula i**ndica o **fim de um comando** ou de **uma declaração**.
- ✓ O Compilador Dev-C++ faz **distinção entre letras maiúsculas e minúsculas**, ou seja, ele é ***case sensitive***.
- ✓ A função **main()** é o Programa Principal, sendo as declarações e comandos nela contidas, delimitadas por **{** (início do bloco de comandos) e **}** (fim do bloco).
- ✓ A linguagem **C++ é insensível a espaços em branco.** Portanto, **um programa poderia ser todo escrito em uma única linha,** desde que sintaticamente correto e respeitando o **limite de 127 caracteres por linha**.
- ✓ Para inserir **comentários** dentro do programa, isto deve ser feito, colocando os comentários entre **chaves** ***/* */*** ou após um ***//***.

  /* */ é usado para comentário de mais de uma linha.

  // é usado para comentário em uma única linha.

Além disso, **quando estiver programando/editando o programa, procurar sempre:**

- ✓ **Indenta-lo**, ou seja, utilizar recuos de textos principalmente em estruturas de controle.

- ✓ **Utilizar-se de um padrão para nomear:** Programas, Variáveis, Constantes, Procedimentos e Funções.
- ✓ **Efetuar testes exaustivos do programa**, preferencialmente na fase de desenvolvimento do algoritmo.

Lembrete!!!

Construa o seu programa pensando que este poderá ser lido, sem maiores dificuldades, por outros programadores nem sempre próximos de você, e que pensam e podem adotar estilos de programação diferente do seu.

2.6 O ambiente de desenvolvimento da Linguagem C++

O Dev-C++ é um software composto de várias funcionalidades, que permitem o desenvolvimento e teste de programas escritos na linguagem C++, ou seja, ele contém um **COMPILADOR** para a linguagem C++, também um **EDITOR de textos** interno que permite fazer as edições do programa-fonte sem necessidade do auxílio de outros editores de textos.

A **IDE Dev-C++** ou **Ambiente Integrado de desenvolvimento da linguagem C++** utiliza os compiladores do projeto GNU (lançado em 27 de setembro de 1983 por Richard Stallman e que atualmente é mantido pela FSF (*Free Software Foundation*)), para desenvolver programas para a plataforma MS-Windows®. Portanto, o Dev-C++ é a ferramenta responsável pela integração de Editor e Compilador em uma única interface (tela), no caso, uma interface MS-Windows®. Além desses componentes a IDE do C++ apresenta outras funcionalidades importantes, acessadas através da barra de Menu.

☞ O Ambiente de Desenvolvimento do Dev-C++ possui, entre outros recursos, o seguinte:

- ✓ **Editor de Textos;**
- ✓ **Abertura de vários programas fontes (.PAS), ao mesmo tempo,** sendo apresentados através de abas de janelas**;**
- ✓ **Utilização do mouse;**
- ✓ **Help de instruções, comandos, funções e procedimentos**
- ✓ **Compilador C++;**
- ✓ **Link-Editor e Gerador de código executável (.EXE).**

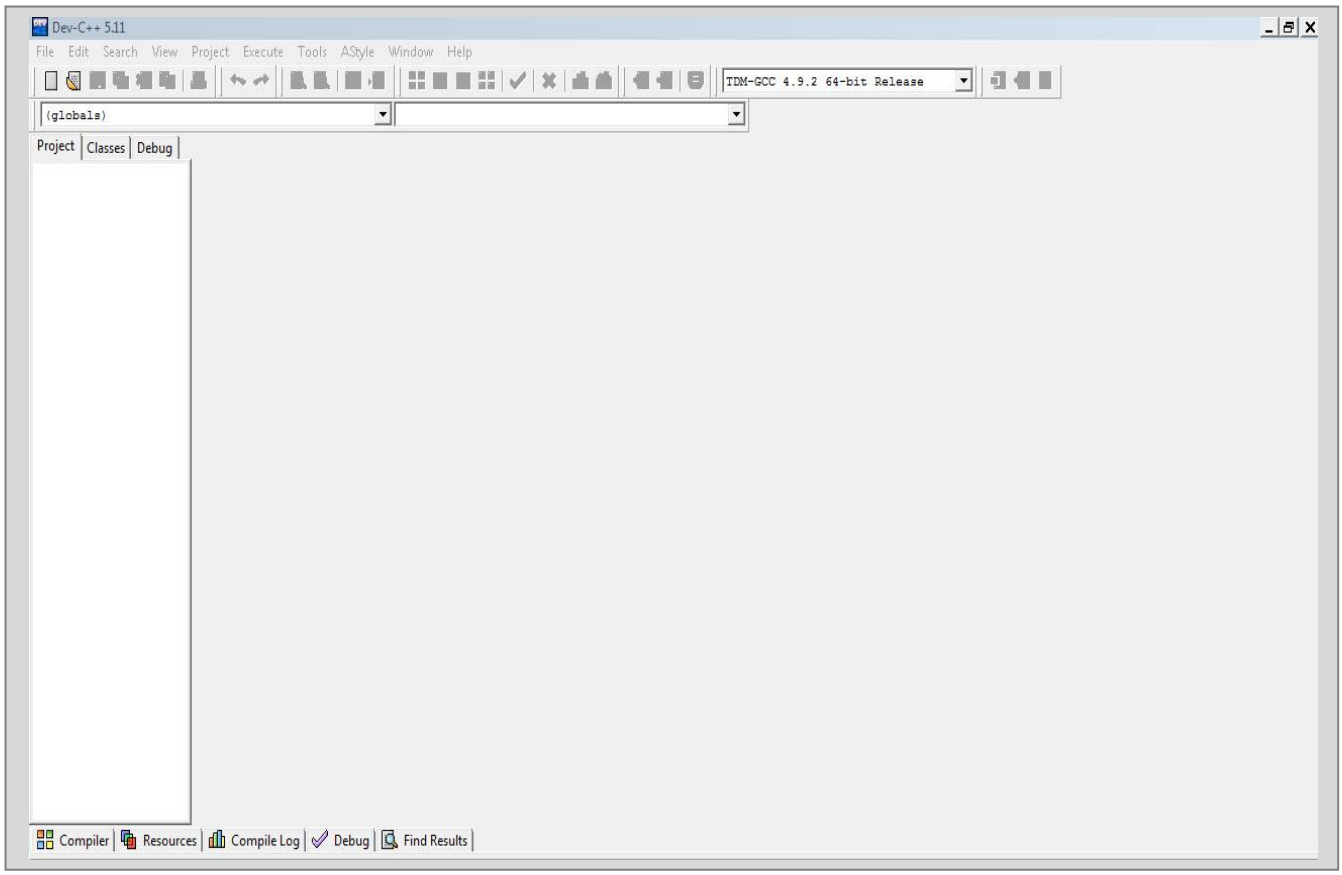
Atenção!

para efetuar o *download* do Dev-C++ acesse o site https://sourceforge.net/projects/orwelldevcpp/ e siga as instruções contidas nele. Ao efetuar o *download* considere qual o sistema operacional está instalado no seu computador.

2.7 Acessando a IDE do Dev-C++

Para acessar o ambiente de desenvolvimento de programas em C++, o Dev-C++, basta clicar no Ícone referente ao software, que você criou como atalho na interface (área de trabalho) do MS-Windows®. Lembre-se que ao criar o atalho, este deve apontar para o seguinte *path* (caminho) do seu HD (hard Disk): **C:\Dev-Cpp\devcpp.exe**.

Quando for executado esse procedimento, será exibida a interface abaixo, que é o ambiente onde o programador irá desenvolver seus programas (algoritmos previamente elaborados).



A partir dessa interface poderá ser criado **(New)** programas-fontes novos, ou abertos **(Open)** programas-fontes já editados anteriormente.

2.8 Etapas do desenvolvimento de um programa em C++

Para desenvolver um programa em linguagem C++ o programador precisa: conhecer a estrutura e a sintaxe dessa linguagem; ter acesso ao compilador da linguagem; familiarizar-se com essa interface (tela) do ambiente Dev-C++.

O processo de desenvolvimento pode ser dividido nas seguintes etapas:

- **Edição do Programa**

  Esta etapa consiste em utilizar as técnicas de construção de algoritmo já estudadas, escrevendo o algoritmo em papel, testando-o logicamente, e em seguida digitando o programa usando o editor de texto contido no ambiente Dev-C++ (opções: ***File*** e ***Edit***).

Nesta etapa estamos construindo o **Programa Fonte**, que **terá um nome** (de arquivo) **e uma extensão** (**.cpp**) que será dada automaticamente pelo editor Dev-C++, ao salvar (***Save*** ou ***Save As***) o arquivo.

Exemplo: **PROG123.cpp**

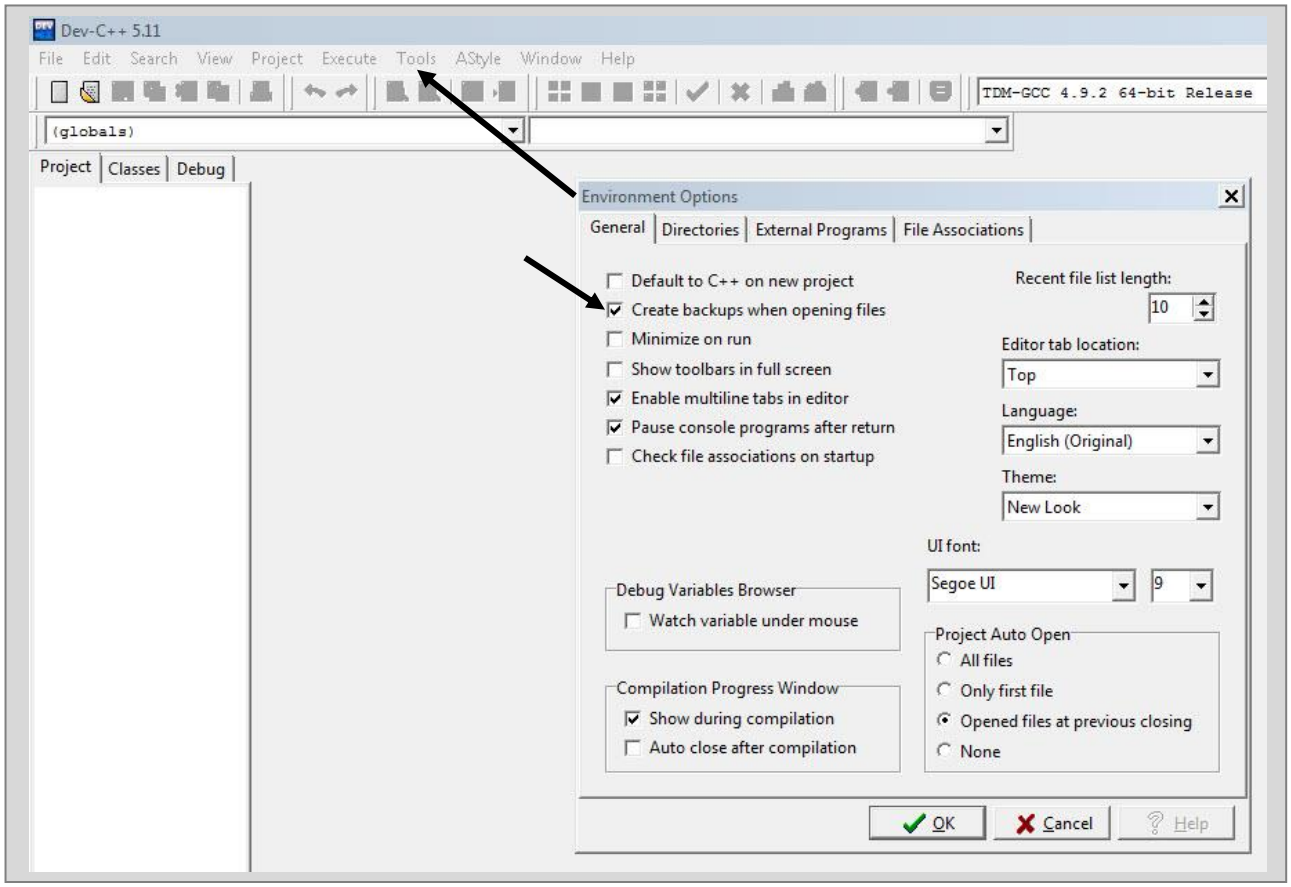
Observação: Quando editamos um programa fonte já existente em algum diretório, o Dev-C++, poderá, automaticamente, **salvar a versão original** do programa, criando um arquivo com o mesmo nome do programa fonte + uma sequência aleatória de números.

Exemplo: **PROG123.1503779576.cpp**

Atenção! Normalmente não é criado backup do programa-fonte que está sendo alterado. Para que isso seja feito, deverá ser efetuada uma alteração na configuração de ambiente no Dev-C++.

Para fazer isso, acesse a opção: ***TOOLS***, e selecione: ***Environment Options***, irá aparecer a janela de opções de ambiente, então marque a opção: ***Create backups when opening files***. Pressione **OK**, e toda vez que alterar um programa-fonte será criado um backup.

Exemplo:



- **Compilação do Programa**

Nesta etapa, após o **Programa Fonte** ter sido digitado, este será compilado, e serão apontados, pelo compilador, os erros de sintaxe cometidos pelo programador. O compilador aponta qual linha/coluna que foi encontrado o erro.

Cada erro apontado deverá ser corrigido, e o programa compilado novamente, até que todos os erros sejam eliminados.

Como resultado do processo de compilação, desde que o programa fonte não tenha apresentado erros de sintaxe, **tem-se um programa** pré- compilado **(Programa Objeto**), com o mesmo nome do programa-fonte, porém com extensão: **.O**.

O processo seguinte à geração do Programa Objeto é chamado de **Link-Edição ou Montagem do programa**, e consiste em agregar ao Programa Objeto, os recursos contidos em bibliotecas de procedimentos e funções do C++, utilizados (codificados) pelo programador em seu programa-fonte.

No ambiente Dev-C++ a opção: ***EXECUTE***, permite que seja escolhido:

- ✓ Somente compilar o programa: ***COMPILE*** **(**ou tecla **F9)**.
- ✓ Somente executar um programa já compilado: ***RUN*** **(**ou tecla **F10).**
- ✓ Compilar e executar um programa, desde que o mesmo não apresente erros de sintaxe: ***Compile & Run*** **(**ou tecla **F11).**

Observação:

Usando a IDE do Dev-C++, pode-se notar que o programa será compilado **"em memória",** e ficará disponível para execução imediata. Neste caso o **programa executável ficando na memória**, ele deixará de existir no momento que for escolhida a opção/comando **file/exit**, para sair do ambiente de desenvolvimento.

Todavia, o programa executável ficará no diretório escolhido para guardar os arquivos gerados no ambiente, e poderá ser executado a qualquer momento, bastando clicar sobre o nome do arquivo executável (.exe).

IMPORTANTE! - para familiarizar-se com o ambiente Dev-C++ aconselha-se iniciar a construção de programas e, constantemente, acessar esse ambiente, desenvolver novos programas, explorando suas funcionalidades.

Somente com muitos exercícios o processo de desenvolvimento de programas em linguagem C++ ficará sedimentado.

2.9 O que é uma boa linguagem de programação de computador?

Essa é uma pergunta que sempre permeia as discussões entre programadores de computador, e grandes discussões são geradas em torno disso, pois linguagem de programação é uma ferramenta, e como todas as ferramentas elas são desenhadas para determinadas finalidades, e com o tempo podem ser substituídas por outras, ou simplesmente podem deixar de existir. Diante disso, pode-se perceber que discussões nesse contexto poderá trazer pouca, ou nenhuma contribuição, pois a escolha de determinada linguagem depende muito do programador, de seus objetivos e também dos conhecimentos prévios sobre outras linguagens e, principalmente, sobre o desenvolvimento de algoritmos.

Todavia, algumas características devem ser levadas em consideração para a escolha de uma boa linguagem de programação. Portanto, uma boa linguagem de programação é aquela que:

- ✓ **Possui uma extensiva capacidade de gerenciamento de arquivos e bancos de dados, com rapidez, integridade e segurança;**
- ✓ **Trata arquivos independentemente da quantidade de registros, sem perder performance (agilidade);**
- ✓ **Tem compatibilidade com outras bases de dados;**
- ✓ **Tenha portabilidade, ou seja, possa ser executada em qualquer plataforma de máquina, sem necessidade de modificações ou recompilações;**
- ✓ **Possa ser utilizada (programada) em vários ambientes;**
- ✓ **Transmite confiabilidade e mantém continuidade (novas versões);**
- ✓ **Proporciona, através de seu fabricante, um bom suporte técnico, e que possui uma vasta e boa documentação técnica e literária;**
- ✓ **Possui uma interface gráfica amigável (GUI), e finalmente;**
- ✓ **Melhor satisfaz as necessidades do programador de computador.**

2.10 Um primeiro programa em linguagem C++ – Comandos de Entrada e Saída

Para o desenvolvimento/teste de um primeiro programa em linguagem C++ torna-se necessário o aprendizado da sintaxe dos comandos básicos de Entrada e Saída de dados, disponível nessa linguagem, e que permitem uma maior interação entre o usuário e o programa que está sendo executado. **Essa interação ocorre, normalmente, através do teclado e vídeo (console)**.

☞ Comando de **SAÍDA** de dados - ***printf()***

Embora referenciado com sendo um comando, na realidade o ***printf()*** é uma **função que possibilita a saída de valores** (constantes, variáveis ou resultados de expressões) em dispositivos como: **console** e **impressoras**.

Essa função possui a seguinte sintaxe:

printf("string de controle", lista de argumentos);

Uma ***string de controle*** consiste de três elementos:

1. **Caracteres diversos que serão impressos na tela**, ou seja, texto ou cadeia de caracteres (*strings*) que se deseja exibir no dispositivo de saída de dados (console ou impressora).

2. **Caracteres especiais de escape "\"**, utilizados para auxiliar na movimentação do cursor na tela, ou do carro de impressão. São exemplos de caracteres especiais de escape, os contidos na figura abaixo.

Código	Significado
\a	Sinal sonoro ("beep")
\b	Retrocesso ("backspace")
\n	Mudança de linha ("new line")
\f	Avanço de página ("form feed")
\r	Retorno do carro da impressora
\t	Tabulação horizontal ("tab")
\v	Tabulação vertical
\0	Nulo (0 em decimal)
\\	Impressão da barra invertida
\"	Impressão das aspas
\'	Impressão do apóstrofo
\?	Impressão do ponto de interrogação
\N	Constante octal (N é o valor ASCII em octal)
\xN	Constante hexadecimal (N é o valor ASCII em hexa)

3. **Caracteres especificadores de formato "%"**, que definem a posição e a maneira como cada uma das variáveis contidas na ***lista de argumentos***, serão exibidas. São exemplos de especificadores de formato, os contidos na figura abaixo.

Especificador	Tipo do Argumento	Descrição
%d	int	Valor inteiro decimal
%i	int	Valor inteiro decimal
%o	int	Valor inteiro octal
%x	int	Valor inteiro hexadecimal
%X	int	Valor inteiro hexadecimal
%u	int	Valor inteiro decimal sem sinal (*unsigned int*)
%c	int	Um caracter em formato ASCII (código binário correspondente)
%s	char	Uma cadeia de caracteres (*string*) terminada em "\0"
%f	float	Valor em ponto flutuante no formato [-]m.dddddd, onde o Nº de d's é dado pela precisão (padrão é 6)
%e	float ou double	Valor em ponto flutuante em notação exponencial no formato [-]m.ddddd e±xx, onde o Nº de d's é dado pela precisão
%E	float ou double	Valor em ponto flutuante em notação exponencial no formato [-]m.ddddd E±xx, onde o Nº de d's é dado pela precisão
%g	float ou double	Valor em ponto flutuante no formato %e (quando o expoente for menor que -4 ou igual a precisão) ou %f (nos demais casos). Zeros adicionais e um ponto decimal final não são impressos
%G	float ou double	Valor em ponto flutuante no formato %E (quando o expoente for menor que -4 ou igual a precisão) ou %f (nos demais casos). Zeros adicionais e um ponto decimal final não são impressos
%%	-	Exibe o caracter '%'

Portanto, a função **printf()** converte e imprime seus argumentos na saída padrão (console, ou impressora) seguindo o formato especificado na *string de controle.*

☞ **Exemplo de programa em C++ usando a função *printf()*:**

```
#include<stdio.h>
#include<stdlib.h>

main()
{
   // Definição de variáveis

   int num1, resultado;

   printf("\n << Operadores de Incremento e Decremento >>");

   num1 =120;
   resultado = ++num1;
   printf("\n\n Resultado ++var %d ",resultado);

   num1 = 120;
   resultado = num1++;
   printf("\n\n Resultado var++ %d num1 = %d",resultado,num1);

   num1 = 120;
   resultado = --num1;
   printf("\n\n Resultado --var %d ",resultado);

   num1 = 120;
   resultado = num1--;
   printf("\n\n Resultado var-- = %d num1 = %d ",resultado,num1);

   printf("\n");

   return 0;
}
```

☞ Comando de **ENTRADA** de dados ***- scanf()***

Embora referenciado com sendo um comando, na realidade o ***scanf()*** é uma **função que lê caracteres da entrada padrão (teclado),** interpreta-os segundo a especificação de formato (**string de controle**), e armazena os resultados em variáveis declaradas no programa.

Essa função possui a seguinte sintaxe:

scanf ("string de controle", lista de endereços de memória das variáveis);

A representação do **endereço de memória das variáveis** que receberão os dados de entrada é fornecida pelo operador **"&"**, seguido pelo nome da variável declarada no programa.

A ***string de controle*** desta função consiste em:

1. **Caracteres especiais de escape "\",** utilizados para auxiliar na movimentação do cursor na tela ou do carro de impressão, conforme já mostrados anteriormente.
2. **Caracteres especificadores de formato "%",** que definem a posição e a maneira como as variáveis serão inicializadas. Exemplos de caracteres especificadores de formatos, conforme já mostrados anteriormente:

Especificador	Descrição
%d	Indica um valor inteiro decimal
%i	Indica um valor inteiro (podendo estar na base decimal, octal com inicial **0** (zero) ou hexadecimal com inicial **0X**
%u	Indica um valor inteiro decimal sem sinal
%c	Indica um caracter (*char*)
%s	Indica uma *string*
%f, %e, %g	Indica um valor em ponto flutuante de precisão simples (*float*)
%lf, %le, %lg	Indica um valor em ponto flutuante de precisão dupla (*double*)
%o	Indica um valor inteiro octal (com ou sem zero inicial)
%x ou %X	Indica um valor inteiro hexadecimal (com ou sem "0x" inicial)

☞ **Exemplo de programa em C++ usando a função *scanf()*:**

```
#include<stdio.h>
#include<stdlib.h>

int main()
{
   // Definição de variáveis
   float largura, comprimento, valor_m2, area_total, val_terreno;

   system("cls");

   printf("\n<< Programa para calcular a Área Total e o Valor de um Terreno Retangular >>");
   printf("\nDigite os seguinte dados do Terreno:");

   printf("\n\nLargura.....: ");
   scanf("%f", &largura);
   printf("Comprimento.: ");
   scanf("%f", &comprimento);
   printf("Valor do M2.: ");
   scanf("%f", &valor_m2);

   area_total = largura * comprimento;                    // calcula área do terreno
   val_terreno = area_total * valor_m2;                   // calcula o valor do terreno

   printf("\n\nÁrea Total do Terreno: %.2f - Valor do M2 R$ %.2f",area_total, valor_m2);
   printf("\n\nValor do Terreno R$ %.2f\n", val_terreno);

   return 0;
}
```